Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

This project will use concepts like functions, routes, conditions, lists, dataclasses, and state variables. I expect to use lists to store swim sets, conditionals to generate different workouts based on user input and loops to display workout information. I want to improve my understanding of state variables while I become more comfortable building a larger project.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I think the hardest part will be transforming my swimming logic into the computer so that it generates workouts that are appropriate for different skill levels while also keeping the total workout distance wanted in mind. The easiest part will most likely be creating the pages and organizing the information since I have a good understanding of swim workouts.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

The workout generation logic will likely require the most time because it involves picking workouts based on multiple user inputs/preferences. I think my first step will be creating the dataclasses and state variables. Then I can build the basic pages and navigation.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

My biggest distraction will be other coursework from classes. To minimize distractions, I plan to work on the project in small sections, a little bit each day, rather than waiting until the deadline. Some resources I plan on using are the textbook, notes, Drafter documentation and office hours. One thing I think could be helpful is seeing a previous example from a student.

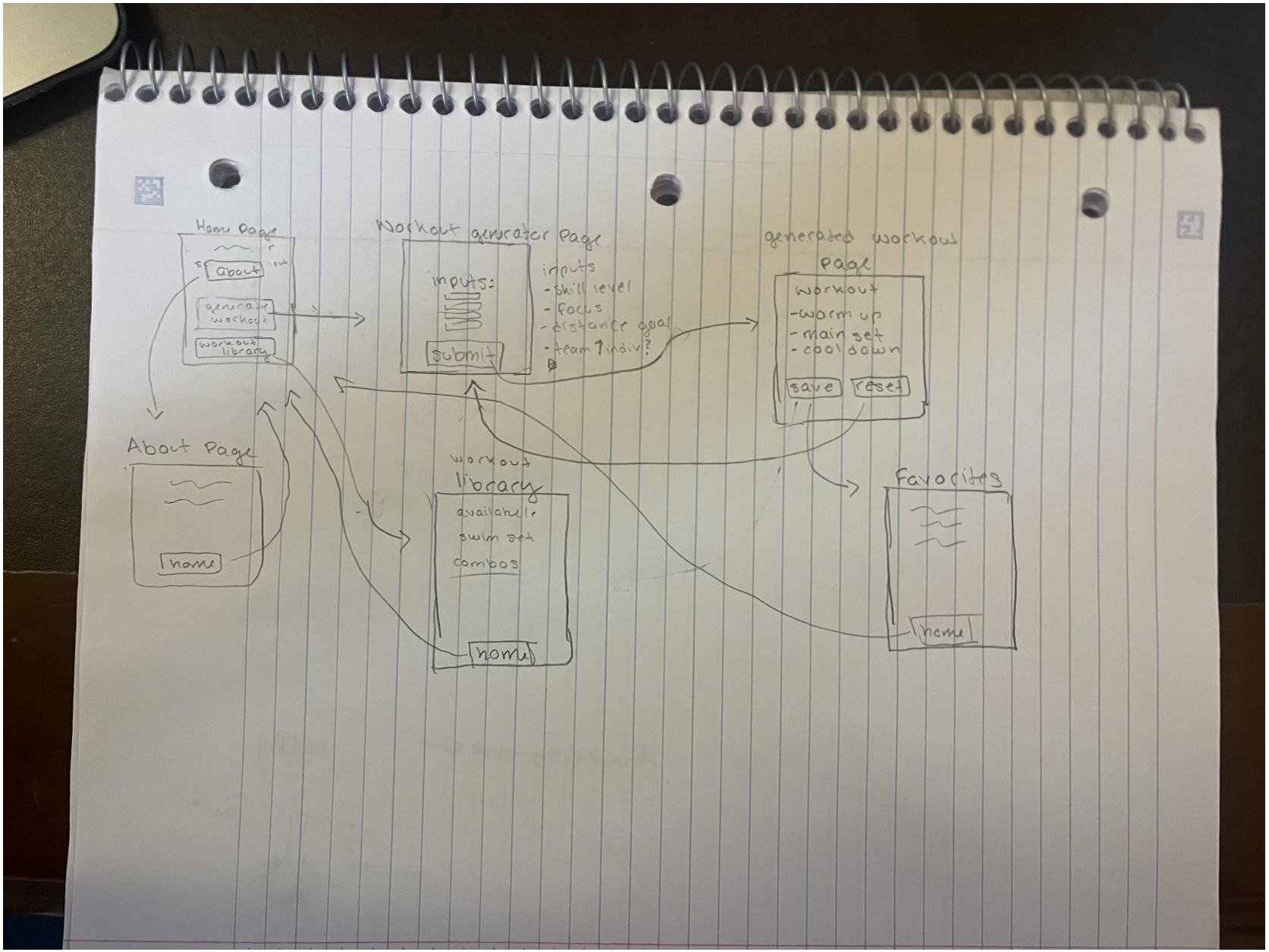
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.


Home Page
about
generate workout
workout library
Workout generator Page
inputs:
submit
inputs
- skill level
- focus
- distance goal
- team? indiv?
generated workout Page
workout
-warm up
- main set
- cool down
save
reset
About Page
home
workout library
available swim set combos
home
Favorites
home

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	The website will help swimmers quickly create structured workouts without spending time planning practices themselves. It generates a warmup, main set and cooldown based on the user's preferences.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	1. skill_level: str → updated when the user selects a skill lever (beginner, intermediate, pro) 2. workout_focus: str → updated when the user selects a training focus (endurance, pace training, sprint, technique/drill, recovery) 3. distance_goal: int → updates when the user enters their desired workout distance 4. current_workout: Workout → updates when all inputs are in and and a new workout is generated for the user 5. favorite_workouts: list[Workout] → updates when the user saves a workout to favorites
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	Workout- represents individual workout components that can be combined to create a complete practice 1. warmup: list[str] → list of the warmup sets 2. main_set: list[str] → list of the main sets of the workout 3. cooldown: list[str] → list of cooldown swims for the swimmer to complete 4. total_distance: int → distance of set
Routes (minimum 7) The names of your route	1. home → main landing page 2. workout_generator → page where user inputs information 3. generate_workout → creates the workout

functions	based on inputs 4. view_workout → displays generated workout to user 5. workout_library → displays all available workout ideas and combinations to user 6. favorites→ displayed saved workouts 7. save_favorite → saves workout to favorites 8. about → displays info about the website *** (might not need this)
Pages (minimum 5) A brief description of the pages in your site	- Home: main page with navigation linked - Workout Generator: where the user enters their workout preferences - Generated Workout: shows warmup, main set, cooldown, and total distanced based on user preferences - Workout Library: shows example swim sets and workout ideas - Favorites: shows workouts saved by the user - About: explains the purpose of the website and how it works ***
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	SelectBox: Skill Level → user chooses between beginner, intermediate, or advanced SelectBox: Workout Focus → user chooses between endurance, sprint, pace training, technique, or recovery TextBox: Distance Goal → user enters desired workout distance ***CheckBox: Competitive Swimmer (might not keep this because if I did, it would probably involve times)
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	if skill_level == “beginner” - Will generate shorter and less challenging workouts - Changes for each input if workout_focus == “sprint” - Will generate sprint-focused sets with shorter repeats - Etc if different input

	if workout_focus == “endurance” - Generates longer aerobic sets, etc if distance_goal >2500 - Adds more sets to increase workout volume
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	I am still workshopping a loop but: - A loop will iterate through the list of workouts saved in favorite_workouts. It will display each saved workout on the favorites page, including the warmup, main set, cooldown, and total distance. It is meaningful because it will allow the users to view all of their saved workouts.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

what is the difference between my state dataclass and my second dataclass and how do i implement my second dataclass into my state one

The main difference is that your State dataclass and your second dataclass serve two different purposes:

- State dataclass: Holds the overall information your application needs to run. It acts as the central container for your app.
- Second dataclass: Represents a specific object or type of information your application needs to keep track of.

For example, imagine you are building a study tracker:

```
@dataclass
class StudySession:
    subject: str
    hours: float
```

```
    date: str
```

This second dataclass represents one individual study session.

Your State dataclass could then contain a list of those sessions:

```
@dataclass
class State:
    sessions: list[StudySession]
```

Now your State keeps track of all the StudySession objects.

How they work together

When a user creates a new study session, you add it to the list inside State:

```
def add_session(state, subject, hours, date):
    new_session = StudySession(subject, hours, date)
    state.sessions.append(new_session)
    return state
```

This way:

1. `StudySession` defines what an individual session looks like.
2. `State` stores all the sessions.
3. Your functions can add, remove, or analyze sessions through State.

Let's look at your actual code

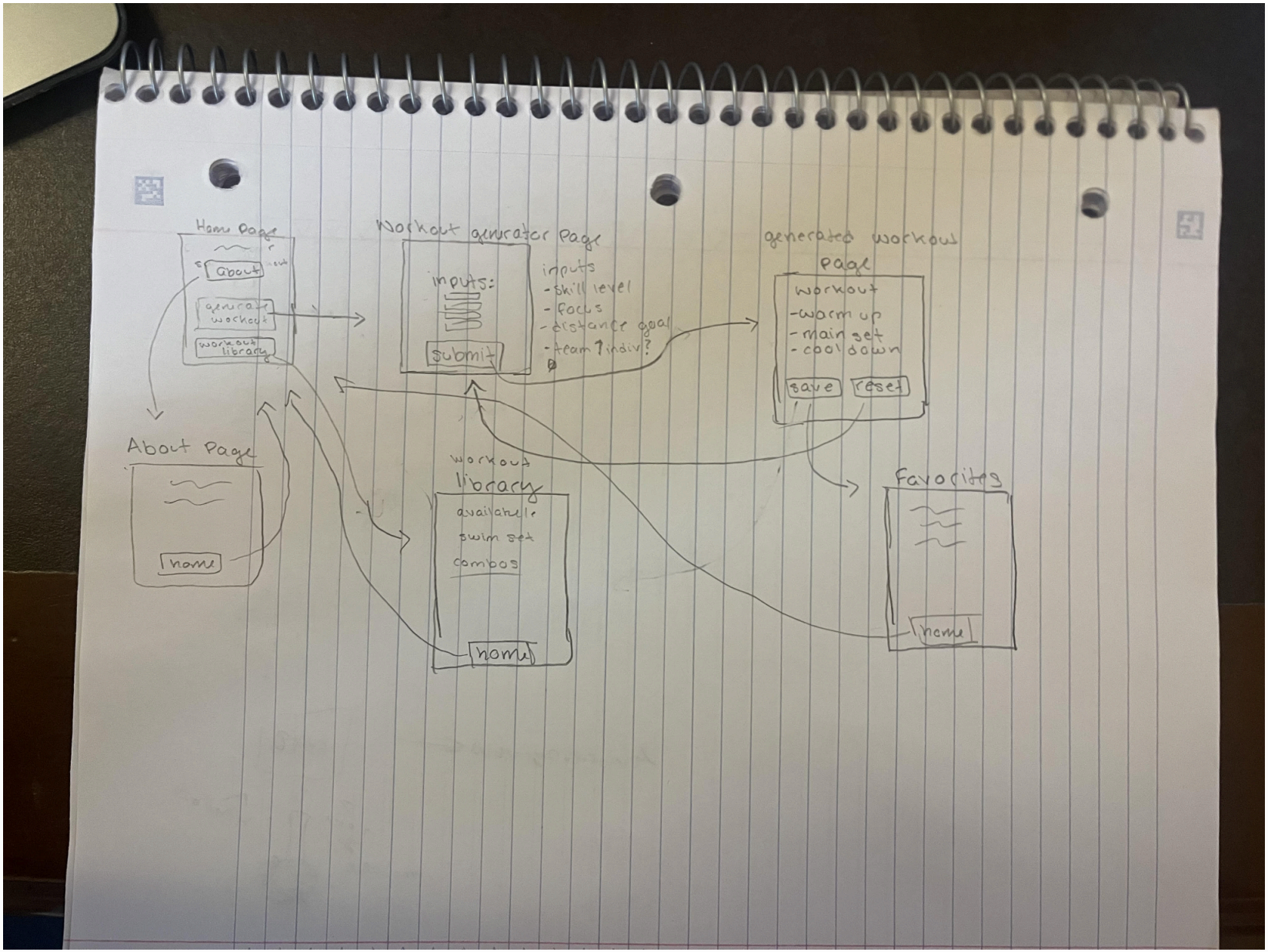
I can explain exactly how to connect your two dataclasses, but I need to see them first.

Paste your State dataclass and your second dataclass here.

I'll explain:

- What each variable in both dataclasses does.
- Why you need two separate dataclasses.
- Which variable you should add to State.
- How to initialize and use your second dataclass in Drafter.

I'll keep the explanation at a CS1 level so you understand how it works rather than just copying code.


Home Page
About
generate workout
workout library
Workout generator Page
inputs:
submit
inputs
- skill level
- focus
- distance goal
- team? indiv?
generated workout page
workout
- warm up
- main set
- cool down
save
reset
About Page
name
workout library
available swim set combos
name
Favorites
name

well im creating both and im not sure how to put them together. basically the goal of my project is to create a website that can generate a swim set based on a users inputs. this is my flow chart

the user will be inputing their skill level, workout focus and distance goal. im pretty sure my second dataclass is supposed hold like swim_set or something that has the warm up main set and cooldown but idk. I dont want you to tell me exactly how to do it though i need to work thorugh it myself

I think you're actually thinking about your second dataclass in the right way! Your idea that it should represent a `SwimSet` containing a warm-up, main set, and cooldown makes sense.

Let's work through the design without me giving you the finished code.

1. First, understand the difference between your two dataclasses

Think about your project as having two different kinds of information:

Dataclass 1

State

Think of this as the website's memory. It keeps track of what is happening in your application.

For example:

- What skill level did the user select?
- What workout focus did they select?
- What distance goal did they enter?
- Has a workout been generated?

Dataclass 2

SwimSet

Think of this as the actual workout that your program creates.

For example:

- Warm-up
- Main set
- Cooldown

The important distinction is:

State stores information about the application, while SwimSet represents the workout your application generates.

2. How do they connect?

Here's the relationship I would encourage you to think about:

USER INPUTS

Skill level · Workout focus · Distance goal

STATE DATACLASS

Stores the user's selections

WORKOUT GENERATION

Uses the inputs to determine the workout

SWIMSET DATACLASS

Contains the warm-up, main set, and cooldown

GENERATED WORKOUT PAGE

Displays the workout to the user

The connection you're looking for is that your State dataclass should have a way to keep track of the generated `SwimSet`.

In other words, you're not combining the two dataclasses into one. You're having one dataclass contain an instance of the other.

3. Let's think through your SwimSet dataclass

I want you to answer three questions before writing it.

Question 1: If your SwimSet contains a warm-up, main set, and cooldown, what Python data type do you think each of those should be?

For example, would each section be:

- A string?
- A list of strings?
- A list of numbers?
- Something else?

Think about how you want to display multiple swimming exercises in each section.

Question 2: Should your State dataclass store the actual warm-up, main set, and cooldown individually, or should it store one `SwimSet` object that contains all three?

Think about which approach avoids unnecessary repetition.

Question 3: What should happen before the user generates a workout?

Your State needs to exist when the website first loads, but the user hasn't generated a workout yet. How might you represent that situation?

Hint: Think about Python's `None`.

4. One thing I noticed in your flowchart

Your overall page flow makes sense. I particularly like that you separated the workout generation page from the generated workout page.

One recommendation: keep your data structure separate from your pages.

For example:

- Your home page collects information.
- Your State remembers that information.
- Your workout-generation function creates a SwimSet.
- Your generated workout page displays that SwimSet.

You don't need a separate dataclass for every page.

Let's start with Question 1: Do you think warm-up, main set, and cooldown should each be strings or lists? Explain why you think your choice makes sense. I'll help you reason through it without writing the solution for you.

well i first thought it could be lists but now im thinking I would need ANOTHER dataclass if i do that

That's actually a really good observation! You're recognizing that if you want each section to contain multiple swimming exercises, a single string might not be enough.

But here's the important thing: You do NOT necessarily need another dataclass just because you're using lists.

Let's compare two possible designs.

Option A: Lists of strings

Imagine a workout looks like this:

```
warmup = ["200 freestyle", "100 kick", "100 pull"]

main_set = ["4 x 100 freestyle", "4 x 50 sprint"]

cooldown = ["100 easy freestyle"]
```

Each section is simply a list of strings.

You wouldn't need another dataclass because Python lists can already hold multiple items.

Option B: Another dataclass

You might eventually want each swimming exercise to have more information.

For example:

- Distance
- Stroke
- Number of repetitions
- Rest time

In that situation, having an additional dataclass representing an individual swimming

exercise could make sense.

However, that adds complexity, and I'm not sure you need it for your CS1 project.

My question for you

Think about what you want your generated workout to look like.

Would you rather have:

A. Simple workout instructions:

- 200 freestyle
- 100 kick
- 4 × 100 freestyle

B. More detailed workout information:

- Distance: 100 yards
- Stroke: Freestyle
- Repetitions: 4
- Rest: 20 seconds

I'd lean toward A for your first version.

Remember, your goal is to build a working swim-set generator, not create an unnecessarily complicated data structure!

yeah for now i think option A is the best. thank you ill work from here and ask more questions if i need any help

You're welcome! I think Option A is a great starting point for your project.

You've got a solid foundation:

- `State` = stores user inputs and the generated workout.
- `SwimSet` = stores the warm-up, main set, and cooldown.
- Lists of strings = store the individual swimming instructions.

Build it one piece at a time, and don't worry about making everything perfect immediately.

Good luck with the project! 😊

if I wanted to include a loop somehow, where would be a good place to implement it?

I actually think there are a few natural places to implement a loop in your swim-set generator! The best place depends on what you want the loop to accomplish.

Here are three possibilities:

1. Generating the individual workout sections

My recommendation

Since you're using lists of strings, you could use a loop to generate or display the exercises in each section.

For example, imagine your main set contains:

```
main_set = ["4 x 100 freestyle", "4 x 50 sprint", "200 kick"]
```

A `for` loop could go through each item and display it on your generated workout page.

Conceptually:

```
for exercise in main_set:

   # Display exercise
```

This is probably the easiest and most natural loop to implement.

2. Displaying the entire workout

You could also use loops to display all three sections:

- Warm-up
- Main set
- Cooldown

This would let you display multiple exercises without writing separate display instructions for every exercise.

3. Calculating the total workout distance

You could use a loop to go through the exercises and calculate the total distance of the workout.

For example, if your workout contains:

- 200 yards
- 100 yards
- 400 yards

- 100 yards

Your loop could add those distances together.

This would be a little more complicated because you would need to extract the distances from your exercise strings.

What I would recommend

For your first implementation, I'd use a `for` loop to display the exercises in your warm-up, main set, and cooldown.

Then, if you want to challenge yourself, you could add a loop to calculate the total workout distance.

One question to think about: Does your assignment require you to demonstrate a loop as part of your program, or are you just looking for a natural way to incorporate one? That might influence which approach makes the most sense.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.